

CLOUDCARE

15-Day Development & Task Allocation Plan

Smart Horizon 2026 | Problem ID SH-FIN-03

Team Alpha — Sumaira · Shruti · Himanshu · Soham · Anay

Sprint window

Wednesday, 15 July 2026 → Thursday, 30 July 2026

Where You Stand Today

This plan is based on the actual repository (cloudcare_practice), not just the concept deck. The Next.js frontend already has a landing page, a login page, and a dashboard shell (KPI cards, cost chart, resource table, agent feed) — currently wired to hardcoded mock data. The FastAPI backend already has routers scaffolded for auth, resources, recommendations, forecasts/savings, and agent runs. The policy engine (safety rules) is already implemented correctly and should not be rewritten. Everything else — auth, orchestrator nodes, AWS collector, executor — is a labeled placeholder returning fake data that needs to become real.

Two Scoping Notes Before You Start

1. "Biometric verification" — use WebAuthn, not custom ML

Building real fingerprint/face recognition is not realistic in 15 days for a beginner team. WebAuthn / passkeys calls the device's own Face ID / Windows Hello / Touch ID and returns a cryptographic assertion to your backend — a well-documented browser API, not something you train. It looks and feels like genuine biometric auth to judges. The plan below scopes auth around this.

2. "Agents optimizing resources" — simulate execution, don't touch real infrastructure

Use a free-tier AWS sandbox account with a read-only IAM role for data collection, and run the Executor in simulation/dry-run mode. It still writes a real, auditable "would have executed X" record — this matches your own blueprint's safety design and removes any risk of a live demo breaking real infrastructure.

Team Roles

Roles are assigned by track, not fixed skill — swap people around based on actual strengths.

Member	Track	Primary Focus
Sumaira	Backend & Data	FastAPI, MongoDB, real CRUD, auth backend, tenant scoping
Shruti	Frontend / Dashboard	Next.js pages & components, live data wiring, UI states
Himanshu	AI / Agents	LangGraph orchestrator, analyzer rules, forecasting, LLM decision node
Soham	Cloud & DevOps	AWS sandbox, collector, executor/policy wiring, Vercel/Render deploys
Anay	Security & QA	3-factor auth (OTP + WebAuthn biometric), safety tests, demo script

Day-by-Day Task Allocation

Day 1 — Wed, 15 July Setup & Alignment

Member	Tasks
All	30-min kickoff call — confirm role split, set up shared Trello/Notion board, agree on Git branching (protected main, one feature branch per person, PR + 1 review before merge).
Sumaira	Set up MongoDB Atlas (free tier). Add real User, CloudAccount, Resource, Proposal collections per the blueprint's data model (section 4).
Shruti	Audit existing components; list what needs real API wiring vs what is just visual polish.
Himanshu	Read the orchestrator/nodes.py stubs; plan what a "real" monitor → analyze → decide flow should look like, starting with structured mock data.
Soham	Create AWS free-tier sandbox account. Set up an IAM read-only role with an external ID (blueprint 2.2 / 7.2). Confirm Render/Vercel env vars are separated (dev vs prod).
Anay	Decide the 3FA stack: password (bcrypt) → email/SMS OTP → WebAuthn biometric step. Write the auth flow doc so backend and frontend agree on payload shapes.

Days 2–4 — Thu 16 to Sat 18 July Real Data Layer + Auth Core

Member	Tasks
Sumaira	Replace the auth.py placeholder with real user lookup + bcrypt password check + JWT issuance (python-jose is already in requirements.txt). Wire resources.py / recommendations.py to MongoDB instead of mock_data.py.
Shruti	Build the login page UI for the 3-step flow (password → OTP input → biometric prompt). Wire the dashboard's KpiCards / ResourceTable to real backend endpoints, including loading and error states.
Himanshu	Implement the deterministic analyzer rules from blueprint §5.1 (idle EC2, over-provisioned, unattached EBS) against a small set of realistic mock metrics — this can run before real AWS data exists.
Soham	Build the collector service for real: EC2 inventory + CloudWatch metrics pull from the sandbox account (blueprint §9.2 AWS session factory). Keep it strictly read-only.
Anay	Implement the OTP generation/verification endpoint plus email/SMS send. Start WebAuthn registration/login endpoints (webauthn library on backend, @simplewebauthn/browser on frontend).

Days 5–7 — Sun 19 to Tue 21 July Wire Auth End-to-End + Orchestration

Member	Tasks
Sumaira	Add tenant-scoped auth middleware (get_current_user dependency) so every endpoint enforces it. Handle session/JWT refresh.
Shruti	Finish the full login flow UI (password → OTP → fingerprint/Face ID prompt via WebAuthn) with clear success/failure states; connect to Anay's endpoints.
Himanshu	Wire the LangGraph graph (orchestrator/graph.py) so monitor → analyze → decide → supervise actually pass state between real nodes, using Soham's collector output.
Soham	Get Cost Explorer data flowing in; hook the collector output into Himanshu's monitor node.

Anay	Finish WebAuthn end-to-end; test the full 3FA flow live between two people on real devices (biometric behaves differently than assumed on localhost). Start writing safety tests from blueprint §13.2.
Checkpoint — end of Day 7 You should be able to log in with all three factors, and see real AWS sandbox resources (not mock data) in the dashboard, even if recommendations are still simple.	

Days 8–10 — Wed 22 to Fri 24 July Recommendations, Forecasting, Policy Gate

Member	Tasks
Sumaira	Build /v1/recommendations/{id}/approve and /execute endpoints for real, persisting to MongoDB with a full audit trail (blueprint §14.1).
Shruti	Build the Recommendations page (before/after sizing, savings, confidence, rationale) and the Approval Queue page.
Himanshu	Wire the decide node to call an LLM (Claude API) for prioritization and rationale only — never for the execute decision, which stays in the deterministic policy engine. Add the forecasting baseline (moving average / seasonal naive, blueprint §5.2).
Soham	Wire the supervise node to call the existing policy engine (do not rewrite it — it is already correct) and route low-risk dev resources to auto-approve, everything else to human review.
Anay	Write and run the safety tests: production never auto-executes, unknown template is blocked, duplicate execute stays idempotent (blueprint §13.2 has the exact test code — use it).

Days 11–12 — Sat 25 to Sun 26 July Execution (Simulated) + Savings Ledger

Member	Tasks
Sumaira	Build the savings ledger data model — baseline cost, post-action cost, verified savings.
Shruti	Build the Agent Activity live feed (poll or WebSocket) and the Savings Ledger dashboard page with real numbers.
Himanshu + Soham (pair)	Implement one real action template end-to-end — ec2.stop.v1 in simulation mode (blueprint §10.1–10.3): validate, "execute" against the sandbox with a dry-run flag, record the result, run the verify step.
Anay	Cross-team QA pass — click through every page as a fresh user; log every bug to the board.

Days 13–14 — Mon 27 to Tue 28 July Hardening & Polish

Member	Tasks
All	Bug bash — everyone tests everyone else's area for 3–4 hours.
Sumaira	Tighten error handling and validation on all endpoints.
Shruti	UI polish pass — consistent loading/empty/error states, responsive check, remove any leftover mock-data references.
Himanshu	Tune forecast and rule thresholds so demo numbers look sensible, not obviously fake.
Soham	Confirm Vercel + Render production deploys are stable, env vars correct, CORS locked down; run a full deploy dry run.
Anay	Write the demo script using blueprint §16 as a template (idle dev instance auto-scheduled, oversized staging instance routed to human approval). Rehearse the 3FA login live at least 5 times — it's the part most likely to fail live.

Day 15 — Wed 29 to Thu 30 July Final Rehearsal & Submission

Member	Tasks
All	Full run-through of the demo twice, on the actual deployed URLs, not localhost. Freeze code — bug fixes only, no new features. Prepare the pitch deck: problem, differentiator, architecture diagram, live demo, impact/roadmap. Submit.

Standing Rules

- Never let the AI decide whether an action executes — only the policy engine does. Judges will likely ask this exact question; "the LLM never controls execution" is your strongest answer.
- Keep a kill switch — even a simple `EXECUTION_ENABLED=false` environment flag — so a misbehaving demo action can be shut off instantly without a redeploy.
- Run a 15-minute daily standup. With 5 people touching a shared repo across auth, orchestration, and AWS integration, merge conflicts will cost more time than the standup does.